Fraudulent Claim Detection

Developing a predictive model for insurance fraud detection offers significant business value. Fraud investigations currently rely on manual processes such as reviewing claims, calling claimants and conducting background checks, which are time-consuming and inefficient. These delays allow fraud to go undetected whereas legitimate claims face unnecessary scrutiny. Predictive modelling enables early identification of high-risk claims, streamlining fraud investigations, reducing financial losses and improving operational efficiency. It also enhances customer experience by expediting legitimate claims. Ultimately, an effective fraud detection model leads to better decision-making, optimised resource allocation and increased profitability.

The objective is to build a model to classify insurance claims as either fraudulent or legitimate based on historical claim details and customer profiles. By using features such as claim amounts, customer profiles, claim types and approval times, the company aims to predict the claims that are likely to be fraudulent before they are approved.

Goal

- Global Insure aims to enhance its ability to detect fraudulent insurance claims by leveraging historical claim data.
- The company seeks to identify patterns and key indicators that differentiate fraudulent claims from genuine ones.
- By developing a predictive model, it intends to assess the likelihood of fraud in incoming claims, enabling proactive fraud detection and reducing financial losses.

To summarise, find the answer to the following questions:

- How can we analyse historical claim data to detect patterns that indicate fraudulent claims?
- Which features are the most predictive of fraudulent behaviour?
- Based on past data, can we predict the likelihood of fraud for an incoming claim?
- What insights can be drawn from the model that can help in improving the fraud detection process?

Overall, the objective is to build a model to classify insurance claims as either fraudulent or legitimate based on historical claim details and customer profiles. By using features such as claim amounts, customer profiles, claim types and approval times, the company aims to predict the claims that are likely to be fraudulent before they are approved.

Problem Statement

Global Insure, a leading insurance company, processes thousands of claims annually. However, a significant percentage of these claims turn out to be fraudulent, resulting in considerable financial losses. The company's current process for identifying fraudulent claims involves manual inspections, which is time-consuming and inefficient. Fraudulent claims are often detected too late in the process, after the company has already paid out significant amounts. Global Insure wants to improve its fraud detection process using data-driven insights to classify claims as fraudulent or legitimate early in the approval process. This would minimise financial losses and optimise the overall claims handling process.

Business Objective

Global Insure wants to build a model to classify insurance claims as either fraudulent or legitimate based on historical claim details and customer profiles. By using features like claim amounts, customer profiles and claim types, the company aims to predict which claims are likely to be fraudulent before they are approved.
Based on this assignment, you have to answer the following questions:

- How can we analyse historical claim data to detect patterns that indicate fraudulent claims?
- Which features are most predictive of fraudulent behaviour?
- Can we predict the likelihood of fraud for an incoming claim, based on past data?
- What insights can be drawn from the model that can help in improving the fraud detection process?

Assignment Tasks

You need to perform the following steps for successfully completing this assignment:

1. Data Preparation
2. Data Cleaning
3. Train Validation Split 70-30
4. EDA on Training Data
5. EDA on Validation Data (optional)
6. Feature Engineering
7. Model Building
8. Predicting and Model Evaluation

Data Dictionary

The insurance claims data has 40 Columns and 1000 Rows. Following data dictionary provides the description for each column present in dataset:

Column Name	Description
months_as_customer	Represents the duration in months that a customer has been associated with the insurance company.

age	Represents the age of the insured person.
policy_number	Represents a unique identifier for each insurance policy.
policy_bind_date	Represents the date when the insurance policy was initiated.
policy_state	Represents the state where the insurance policy is applicable.
policy_csl	Represents the combined single limit for the insurance policy.
policy_deductable	Represents the amount that the insured person needs to pay before the insurance coverage kicks in.
policy_annual_premium	Represents the yearly cost of the insurance policy.
umbrella_limit	Represents an additional layer of liability coverage provided beyond the limits of the primary insurance policy.
insured_zip	Represents the zip code of the insured person.
insured_sex	Represents the gender of the insured person.
insured_education_level	Represents the highest educational qualification of the insured person.
insured_occupation	Represents the profession or job of the insured person.
insured_hobbies	Represents the hobbies or leisure activities of the insured person.
insured_relationship	Represents the relationship of the insured person to the policyholder.
capital-gains	Represents the profit earned from the sale of assets such as stocks, bonds, or real estate.
capital-loss	Represents the loss incurred from the sale of assets such as stocks, bonds, or real estate.
incident_date	Represents the date when the incident or accident occurred.
incident_type	Represents the category or type of incident that led to the claim.
collision_type	Represents the type of collision that occurred in an accident.
incident_severity	Represents the extent of damage or injury caused by the incident.
authorities_contacted	Represents the authorities or agencies that were contacted after the incident.
incident_state	Represents the state where the incident occurred.
incident_city	Represents the city where the incident occurred.

incident_location	Represents the specific location or address where the incident occurred.
incident_hour_of_the_day	Represents the hour of the day when the incident occurred.
number_of_vehicles_involved	Represents the total number of vehicles involved in the incident.
property_damage	Represents whether there was any damage to property in the incident.
bodily_injuries	Represents the number of bodily injuries resulting from the incident.
witnesses	Represents the number of witnesses present at the scene of the incident.
police_report_available	Represents whether a police report is available for the incident.
total_claim_amount	Represents the total amount claimed by the insured person for the incident.
injury_claim	Represents the amount claimed for injuries sustained in the incident.
property_claim	Represents the amount claimed for property damage in the incident.
vehicle_claim	Represents the amount claimed for vehicle damage in the incident.
auto_make	Represents the manufacturer of the insured vehicle.
auto_model	Represents the specific model of the insured vehicle.
auto_year	Represents the year of manufacture of the insured vehicle.
fraud_reported	Represents whether the claim was reported as fraudulent or not.
_c39	Represents an unknown or unspecified variable.

Guidlines

Data Preprocessing and Feature Engineering,Redundant Values and Columns,Why do we need to handle redundant values and columns?

Redundant data, whether in the form of entire columns or specific values within columns, can negatively impact the performance, interpretability and efficiency of your machine learning models.
The possible reasons for data redundancy are the following:

- Unique identifiers and irrelevant information
- Missing or empty data
- Duplicate or highly correlated features
- Low variance or skewed data
- Illogical or invalid values
- Features with low predictive power

Fixing Datatypes

Why do we need to separate categorical features from numerical ones?

- Machine learning models treat categorical and numerical features differently. Categorical variables often require encoding (one-hot or label encoding), whereas numerical features can be used directly in computations.
- Keeping them separate initially ensures correct transformations without unintended numerical operations on categorical data.

Resampling

Resampling addresses class imbalance, ensuring the model is not biased towards the majority class and can effectively learn patterns from all classes. Class imbalance can lead to:

- Poor learning of the minority class
- High overall accuracy but poor performance on the minority class
- Poor generalisation of unseen data

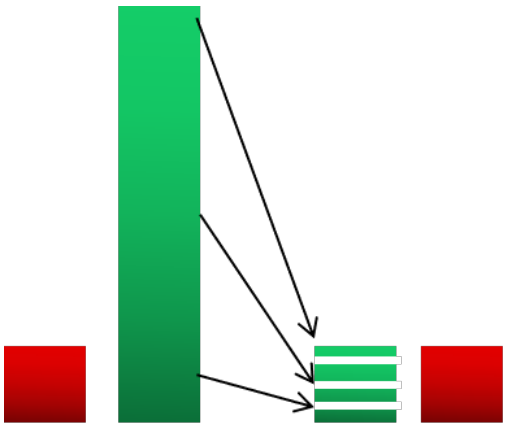
Why do we need to handle the class imbalance?

If one class is under-represented, the model might perform well overall but fail to identify minority class patterns accurately. Resampling helps the model generalise better, especially for minority class predictions.

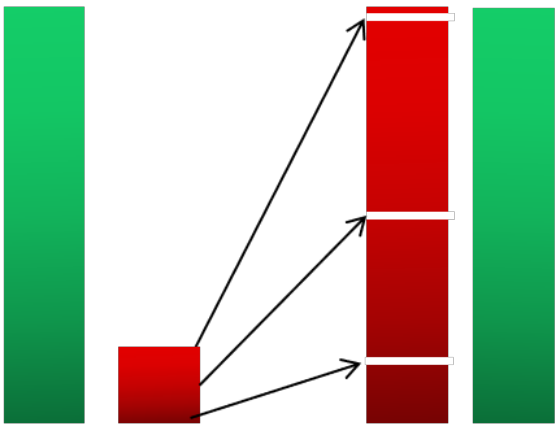
How to perform resampling?

You can use different resampling techniques to handle the class imbalance, and they serve different purposes.

1. **Random undersampling:** In this method, you select fewer data points from the majority class to balance the classes. If the minority class has 500 data points, you would take 500 from the majority class, creating balance. However, this approach is often ineffective as it discards over 99% of the original data, potentially causing bias. You can learn about it here.
2. **Random oversampling:** Using this method, you can add more observations from the minority class by replication. Although this method does not add any new information, there is no information loss. It may also exaggerate the existing information to a certain extent, leading to the problem of overfitting. You can learn about it here.



Undersampling



Oversampling

You can explore other resampling techniques by accessing the following links:

- Undersampling Techniques
- Oversampling Techniques

NOTE: Always **apply resampling only on the training set**, not on the test set, to avoid data leakage. If applied to the test set, the model may memorise test data, leading to misleading performance. Split the data into train and test sets first before handling class imbalance.

Feature Creation

Why do we need to create a new feature?

Creating new features enhances the information available to the model, helping it capture complex relationships and patterns that may not be obvious from the original data. Well-designed features can introduce domain knowledge and improve the model's ability to distinguish between classes, leading to better performance.

You can create a new feature by:

- **Generating interaction terms:** Create new features by multiplying or combining two or more existing features to capture interactions. For example, create a ‘total purchase amount’ by multiplying ‘price’ and ‘quantity’.
- **Extracting information from date/time:** Derive ‘day of the week’ or ‘month’ from a timestamp to capture seasonal trends.

Try using domain knowledge to identify meaningful combinations or transformations, and explore various feature creation strategies to improve your model's ability to learn and generalise.

Combine Values in Categorical Columns

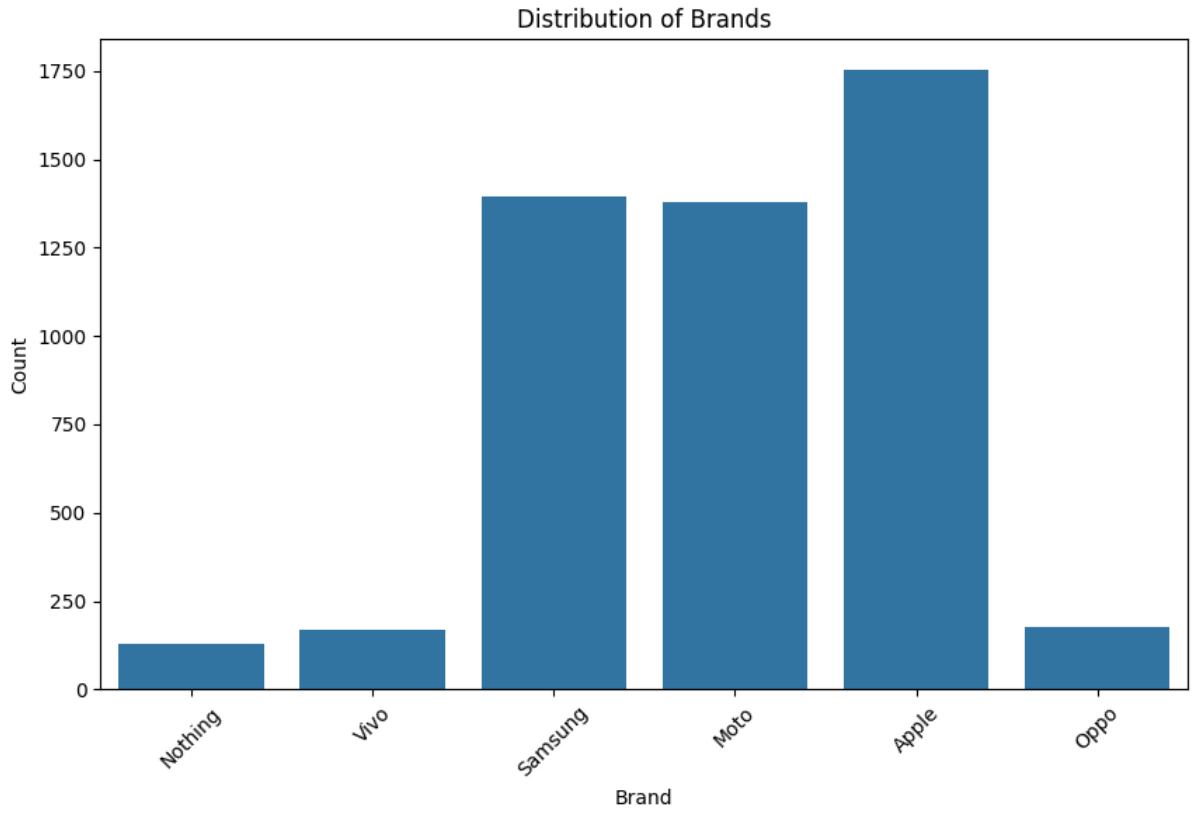
Why do we need to combine or group low-frequency values in categorical columns?

Grouping infrequent categories reduces model complexity by decreasing the number of unique values, making the model easier to interpret and train. It also improves generalisation by helping the model handle unseen or new categories more effectively in future data.

Let’s understand this using an example.

Suppose you have a data set with a categorical column ‘Brand’ containing the following values: Apple, Moto, Samsung, Vivo, Oppo and Nothing.

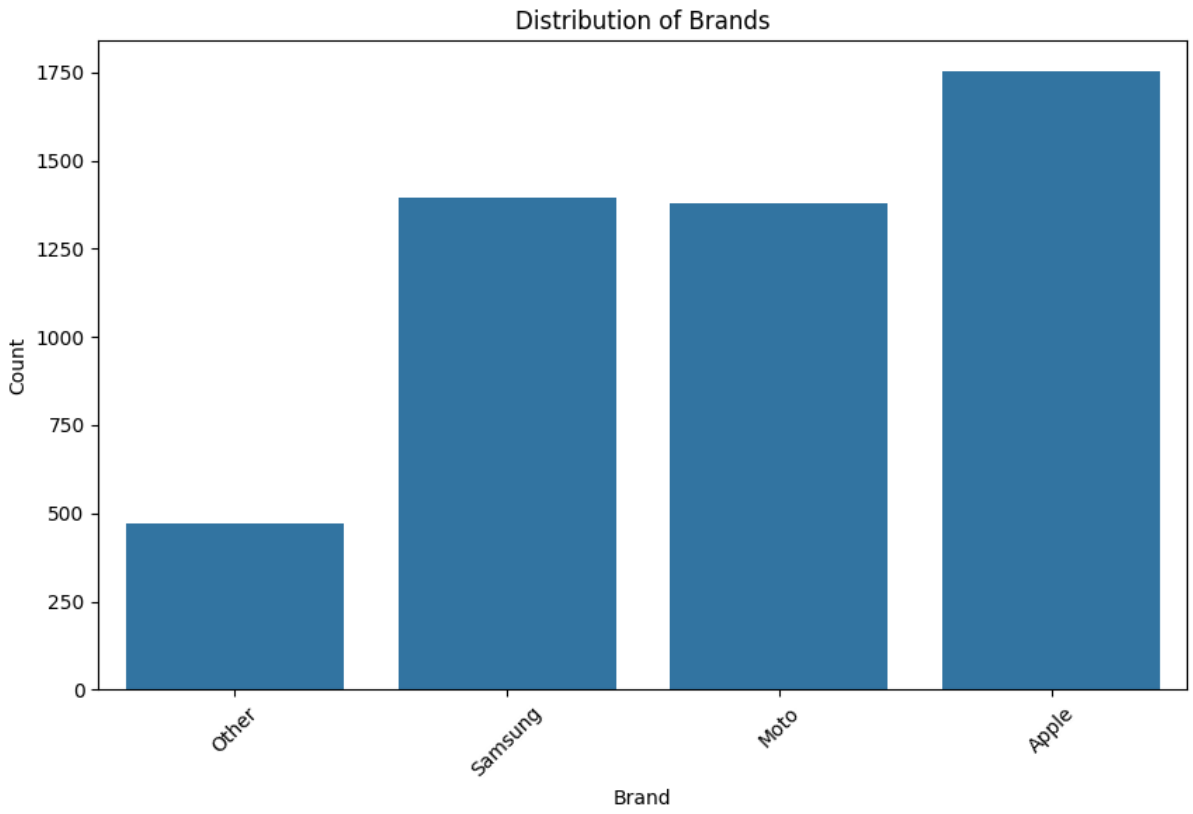
Let's visualise their initial distribution in the graph below.



As you can see, Nothing, Vivo and Oppo appear much less frequently than other categories. Since these low-frequency values are unlikely to contribute much to

the prediction, combining them into a single category called ‘Other’ can help reduce sparsity and improve model generalisation.

Let's combine Nothing, Vivo and Oppo into a single category, ‘Other’ and visualise the updated distribution:



As you can see, the combined ‘Other’ category now has a meaningful count relative to other categories, which helps the model generalise better and reduces the risk of overfitting to rare values.

Feature Scaling

Why do we need to scale the features before model building?
Feature scaling ensures that features with different scales do not disproportionately influence the model's learning process.
For example, if a data set has ‘Product Price’ (ranging from $1 to $1000) and ‘Customer Rating’ (ranging from 1 to 5), scaling ensures that the price feature does not dominate the model's learning process.

EDA

Correlation Analysis
What does correlation analysis reveal about the features?
Correlation analysis of numerical features helps to understand the strength and direction of the relationship between two or more numerical variables. This helps in:

- Detecting highly correlated features, as multicollinearity can affect model interpretation and stability
- Identifying and removing redundant features, improving model efficiency and clarity
- Understanding feature relationships, guiding better feature engineering

You can learn more about it here.

Target Likelihood Analysis

What does the target likelihood analysis reveal about the features?

As the name suggests, the target likelihood analysis examines the relationship between categorical variables and the target variable by calculating the likelihood of the target event for each category of a categorical feature. This helps in:

- Identifying which categorical features have the strongest influence on the target variable
- Revealing how different categories are associated with the target event, offering valuable insights
- Making decisions on feature transformations or combining categories to improve model performance

Model Building

Recursive Feature Elimination (RFECV)

What is RFECV, and how does it select features?

RFECV is an extension of Recursive Feature Elimination (RFE) that automates feature selection by using cross-validation to determine the optimal number of features. It iteratively removes the least important features while refitting a model, ensuring that only the most relevant features are retained. Performance metrics such as R-squared or accuracy guide the selection process, preventing underfitting or overfitting. You can learn more about it here.

How are manual, RFE and RFECV-based feature selections different from one another?

Manual selection: Relies on domain knowledge and exploratory analysis. It is subjective and time-consuming and may miss hidden patterns.

RFE: Iteratively removes the least important features based on model ranking. Requires manual selection of the optimal feature count and lacks cross-validation

RFECV: Extends RFE by using cross-validation to automatically determine the best features, reducing overfitting risks but is computationally expensive

Hyperparameter Tuning

Hyperparameter tuning optimises a model's learning process, helping prevent overfitting and underfitting while improving performance on the given data. Since hyperparameters control the model's learning behaviour, fine-tuning them can significantly impact accuracy, precision, recall or other key metrics.

You can learn more about tuning different models using the following links:

Hyperparameter Tuning - Random Forest

Hyperparameter tuning - Decision Tree

Target Likelihood Analysis

I will get stuck for 2-3 days in this stupid thing
What does the target likelihood analysis reveal about the features?
As the name suggests, the target likelihood analysis examines the relationship between categorical variables and the target variable by calculating the likelihood of the target event for each category of a categorical feature. This helps in:

- Identifying which categorical features have the strongest influence on the target variable
- Revealing how different categories are associated with the target event, offering valuable insights
- Making decisions on feature transformations or combining categories to improve model performance

Definition

Target encoding, also known as mean or likelihood encoding, is a technique used to represent categorical variables as numerical values based on the likelihood of the target variable within each category. This approach is supervised, meaning it leverages the target variable's values during encoding.

Here's how it works:

1. Calculate the mean target value for each category:
   For each level of the categorical variable, compute the average value of the target variable for all instances belonging to that category.

2. Replace categories with calculated means:
   Substitute each category in the original variable with its corresponding mean target value.

Example:
Imagine a categorical variable "City" with levels "New York", "Los Angeles", and "Chicago". If the target variable is "Purchase", which is binary (0 or 1), you would calculate the following:

- New York:
  If 60% of customers from New York made a purchase (i.e., the average "Purchase" value for New York is 0.6), then replace "New York" with 0.6 in the dataset.

- Los Angeles:
  If 40% of customers from Los Angeles made a purchase (average "Purchase" is 0.4), then replace "Los Angeles" with 0.4.

- Chicago:
  If 70% of customers from Chicago made a purchase (average "Purchase" is 0.7), then replace "Chicago" with 0.7.

Key points about target encoding:

- Supervised:
  The encoding is based on the target variable, making it a supervised learning technique.

- Replaces categories with probabilities:
  Essentially, each category gets replaced with the probability of the target variable being "true" within that category.

- Useful for machine learning:
  This encoding method can be helpful for algorithms that struggle with categorical data or when you want to incorporate the relationship between the categorical variable and the target into the model's input, according to Towards Data Science.

- Can be susceptible to overfitting:
  If the number of instances per category is small, the calculated means might not be representative of the true underlying probability, leading to overfitting. Techniques like smoothing or adding regularization can help mitigate this issue.

- Alternative to one-hot encoding:

Target encoding is an alternative to one-hot encoding, especially when dealing with high cardinality categorical variables (variables with many unique levels).

Links: Target Encoding

GFG (https://www.geeksforgeeks.org/machine-learning/mean-encoding-machine-learning/]